

AI Agents Bootcamp — Final Proje Genel Rehberi

Türkiye Veri Topluluğu × MultiGroup · AI Agents Bootcamp 2026

Uçtan uca bir AI agent sistemi geliştir, ölç, güvenli hâle getir, yayına al ve portföyüne koy.

BİR BAKIŞTA

Kapsam	Üç projenin ortak kuralları
Projeler	FinSight (Finans) · ShopOps (E-ticaret) · TrialMatch (Sağlık)
Takvim	2 Ekim – 6 Kasım 2026
Kilometre taşları	M0 → M3
Planlanan teslim	6 Kasım 2026, 23:59 · MultiGroup teyidi bekleniyor

İçindekiler

1. Üç Proje Bir Bakışta.....	3
2. Neden "Agent"? Doğru Problemi Seçmek.....	4
3. Ortak Çekirdek: Temel ve Hedef Seviye.....	4
4. Takvim ve Kilometre Taşları.....	6
5. Repo Yapısı.....	7
6. Repoda Yazılı Olarak Cevaplanması Gereken Sorular.....	7
7. Değerlendirme (Eval) Nasıl Kurulur.....	8
8. Güvenlik ve Etik Kurallar (Tüm Projeler).....	8
9. Ortak Değerlendirme Rubriği.....	9
10. Olsa Güzel Olur (Bonus).....	10
11. Yapay Zekâ Destekli Kodlama Hakkında.....	10
12. Teslim.....	10
13. Mülakatta Projeni Anlatmak: Ortak Çerçeve.....	11
Ek A. Ücretsiz / Düşük Maliyetli Teknoloji Seçenekleri.....	11
Ek B. Yardımcı Kaynaklar.....	13

Bu bootcamp'in sonunda senden bir **çalışan AI agent sistemi** bekliyoruz. Sistem bir sektördeki gerçek bir problemi çözmeli; sen de her kararını belgeleyip ne kadar iyi çalıştığını ölçmelisin. Bir mülakatta açıp gösterebileceğin, "neden böyle yaptım" sorusuna sayılarla cevap verebileceğin bir proje olmalı.

Üç sektörde üç proje hazırladık. Birini seç ve eğitim boyunca adım adım geliştir. Bu rehber üç proje için de geçerli ortak kuralları anlatıyor. Her projenin kendi dokümanında senaryo, veri, mimari, iş kuralları ve rubrik ayrıntıları var.

Bu paketi nasıl kullanacaksın?

Önce bu rehberden **Temel ve Hedef** ayrımını, takvimi ve teslim biçimini oku. Beceri rotası için [Beceri_Haritasi.html](#) dosyasını aç ([Beceri_Haritasi.md](#) düzenlenebilir kaynaktır). Sonra [01_FinSight_Finans.pdf](#), [02_ShopOps_Eticaret.pdf](#) ve [03_TrialMatch_Saglik.pdf](#) dosyalarının ilk sayfalarını karşılaştır; seçtiğin projenin 7. bölümündeki kapsamı incele. Tanıtım sunumu [Sunum/Final_Proje_Tanitim.html](#) dosyasıdır. Sunum, dört PDF ve beceri haritasının HTML dosyası aynı [Final_Projeler](#) klasör yapısında paylaşılsa son slayttaki belge bağlantıları açılır.

[Türkiye Veri Topluluğu](#) topluluğun genel adresidir. **Bu adres proje teslim formu veya teknik yazı yükleme sayfası değildir.** Proje teslim kanalı MultiGroup duyurusuyla, TVT yayınına yazı gönderme yolu ise editör/organizasyon duyurusuyla netleşir. Katılımcılar form veya editör yolu açıklanana kadar GitHub reposunu ve yazı taslağını hazırlayabilir; TVT yolu açılmazsa §12'deki herkese açık Medium seçeneğini kullanabilir.

Organizasyon teyidi bekleyen koşullar: teslim formunun bağlantısı, geçme eşiği ve sertifika koşulları, ekip üst sınırı. Son teslim (6 Kasım 2026) ve sonuç duyurusu (1 Aralık 2026'ya kadar) kesindir.

1. Üç Proje Bir Bakışta

	P1 · FinSight	P2 · ShopOps	P3 · TrialMatch
Sektör	Finans / yatırım araştırma	E-ticaret / pazaryeri	Sağlık / klinik araştırma
Problem	Analist bir şirket için ilk notu 1-2 günde hazırlıyor	Destek ekibi aynı soruları tekrar tekrar yanıtlıyor; yöneticiler basit bir metrik için günlerce bekliyor	Hekim, hastasına uygun açık klinik çalışmayı saatlerce arıyor
Ajanın çıktısı	Her sayısı kaynaklı şirket araştırma notu	Sipariş/iade işlemleri + doğal dille operasyon metrikleri	Kriter kriter gerekçelendirilmiş uygun çalışma listesi
Veri	SEC EDGAR (finansal rakamlar + 10-K metinleri; anahtarsız)	Olist Brazilian E-Commerce (gerçek, anonim; 99.441 sipariş, 9 tablo)	ClinicalTrials.gov API v2 (anahtarsız) + sentetik hasta vakaları + TREC eval seti
İmza konular	Agentic RAG, multi-agent + critic, sayısal doğruluk	Tool calling, MCP, Data Agent & semantic layer, human-in-the-loop	Yapısal çıkarım, kriter bazlı akıl yürütme, Responsible AI
Öne çıkan dersler	7, 8, 9, 11	5, 6, 9, 12	3, 7, 11, 12
En zorlu kısım	Uzun ve dağınık finansal metinler; XBRL tag tutarsızlıkları	Yetkilendirme ve yazma aksiyonlarının güvenliği; text-to-SQL doğruluğu	Belirsizliği dürüstçe gösterme; tıbbi terminoloji (TR↔EN)

	P1 · FinSight	P2 · ShopOps	P3 · TrialMatch
Kime uygun?	Finans, analitik, RAG'e ilgi duyanlar	Yazılım, backend, ürün/operasyon tarafındakiler	Sağlık, NLP, güvenli AI'ya ilgi duyanlar
Temel seviye efor (tahmini)	~40–50 saat	~30–40 saat	~45–55 saat; klinik kriter etiketlemesi sağlık geçmişi olmayanlar için zaman alır

Üç proje de **aynı ortak çekirdeği** (§3) ve **aynı rubrik ağırlıklarını** (§9) kullanır. Hangisini seçersen seç, puanlama adildir. Sektörü merakına ve hedeflediğin işe göre seç. 2 Ekim – 6 Kasım arasında 5 hafta var. Haftada 8–10 saatle ShopOps'un Temel seviyesi rahat biter; FinSight ve TrialMatch için haftada 10–12 saat planla. Hedef seviye her projede haftada 12+ saat ister. Efor tahminleri yaklaşıktır; teknik yazı için ayrıca 4–6 saat ayır.

İpucu: Çalışmak istediğin şirketin sektörüne yakın projeyi seç. Mülakatta "sizin alanınızdaki bir problemi çözdüm" demek, herhangi bir demo göstermekten çok daha güçlüdür.

2. Neden "Agent"? Doğru Problemi Seçmek

Her LLM uygulaması bir agent değildir. Projelerimiz, aşağıdaki özellikler gerçekten gerektiği için agent olarak tasarlandı:

- **Adım sayısı önceden belli değil.** Soru bazen tek bir tool çağrısıyla, bazen beş çağrıyla çözülür. Sırayı model, gözlemlere bakarak belirler (TAO / ReAct — Ders 3).
- **Dış dünyaya erişim gerekiyor.** Veritabanı, API ve doküman olmadan doğru cevap verilemez (Ders 2, 6, 7).
- **Uzmanlaşma işe yarıyor.** Veri toplayan, yazan ve denetleyen ayrı rollere bölmek kaliteyi ölçülebilir şekilde artırır (Ders 8).
- **Hata pahalı.** Bu yüzden ölçüm, izleme ve güvenlik projenin parçasıdır, süsü değildir (Ders 11, 12).

README'nde şu soruyu mutlaka cevapla: **"Bu problem neden sabit bir workflow ya da tek bir LLM çağrısıyla yeterince iyi çözülemiyor?"** Bunu kendi baseline ölçümünle göster (§7).

3. Ortak Çekirdek: Temel ve Hedef Seviye

Aşağıdaki dokuz madde, portföyde güçlü duracak **Hedef** seviyedeki sistemi tanımlar. **Temel** seviye çalışan asgari sürümdür: bazı maddelerin hafif bir hâlini içerir, bazılarını Hedef'e bırakır. Proje dokümanlarının 7. bölümü her maddenin o projedeki karşılığını anlatır.

#	Gereksinim	Hedef seviyede (tam puan yolu)	Temel seviyede (çalışan asgari sürüm)	İlgili ders
1	Özel tool'lar	En az 4 kendi yazdığın tool. Tipli parametre şeması, modelin anlayacağı açıklama (docstring) ve hata durumunda anlamlı mesaj ("bu sipariş bulunamadı", "tarih formatı YYYY-AA-GG olmalı") olmalı.	Aynısı: zorunlu	2, 5, 6

#	Gereksinim	Hedef seviyede (tam puan yolu)	Temel seviyede (çalışan asgari sürüm)	İlgili ders
2	Agent döngüsü	ReAct / tool-calling döngüsü; max_steps sınırı, aynı tool'u aynı argümanla tekrar tekrar çağırmaya karşı koruma.	Aynısı: zorunlu (tek agent yeterli)	3, 4, 5
3	Bilgi katmanı	RAG ve/veya yapısal veri sorgusu. Cevaplarda kaynak gösterme (doküman, bölüm, kayıt kimliği).	Aynısı: zorunlu	7, 9
4	MCP	Tool'larından en az birini kendi yazdığın bir MCP server olarak sun; agent da onu MCP client olarak kullansın.	Hedef'e bırakılabilir	2, 6
5	Multi-agent	En az 2 uzman agent + bir orkestratör/router. Her agent'ın sorumluluğu ve tool seti ayrı tanımlı.	Hedef'e bırakılabilir	8
6	Değerlendirme	En az 25 örneklilik golden set , otomatik metrikler + LLM-as-judge, baseline vs final tablosu, en az 5 başarısız örnekle hata analizi.	≥25 örneklilik golden set + baseline vs final tablosu (LLM-as-judge ve hata analizi Hedef'te)	11
7	Observability	Her çalıştırmanın trace'i (adımlar, tool çağrıları, girdi/çıkıtlar); adım başına token, maliyet ve gecikme.	Adımların ve token sayısının loglanması yeterli	11
8	Güvenlik & Responsible AI	Prompt-injection test seti (≥5), veri yazan/değiştiren aksiyonlarda insan onayı (HITL), sır ve kişisel veri yönetimi, kapsam dışı taleplerde reddetme.	Kapsam dışı reddetme + yazma aksiyonu varsa HITL + anahtarlar repoda değil + projenin 7. bölümündeki zorunlu güvenlik testleri	12
9	Deploy	Arayüz (Gradio, Streamlit vb.) + herkese açık demo linki + README'de ekran görüntüsü ya da GIF. Ücretsiz LLM kotası biterse ya da demo uykudaysa kullanıcıya anlaşılır bir mesaj gösterilmeli.	CLI veya notebook demo + README'de ekran görüntüsü	11

Her seviyede zorunlu: README (§5), §6'daki soruların cevapları ve TVT ya da Medium'da yayınlanmış bir teknik yazı (§12). Temel seviyede cevaplanamayan sorular için "bu sürümde yok, çünkü..." yazmak yeterli.

"Golden set örneği" uçtan uca bir görev demektir: bir girdi ve o girdi için beklenen çıktı ya da davranış. TrialMatch'te TREC hasta vakaları uçtan uca örnek sayılır; kriter düzeyindeki etiketli çiftler bunlara ektir.

Öğrenme çıktıları ve agent skill kullanımı

Beceri_Haritasi.md, sekiz ana beceriyi gözlenebilir proje kanıtına ve derslere bağlar. Katılımcı önce tool, ajan döngüsü, kaynaklı bilgi ve ölçümü çalışan bir Temel sürümde gösterir; orkestrasyon, MCP ve açık demo Hedef sürümde derinleşir. Ders 7 için çalışma dosyasında yer alan *skill generation & usage* önerisi, resmî başlık veya yeni bir teslim zorunluluğu değildir. Projede tekrar kullanılabilir bir agent skill yazarsan onu bir tool ya da güvenlik kontrolü yerine sayma; girişini, çıktısını ve golden set üzerindeki etkisini göster.

Teknoloji seçimi serbest, gerekçesi zorunlu

- **Framework:** smolagents, LlamaIndex, LangGraph, Google ADK ya da doğrudan model SDK'sı. Hepsini kabul edilir. Neden seçtiğini README'de 3-5 cümleyle anlat. Birden fazlasını birleştirebilirsin (ör. LlamaIndex ile RAG, LangGraph ile orkestrasyon).
- **LLM:** İstedikini sağlayıcı. Ancak projen **ücretsiz bir seçenekle de uçtan uca çalışabilmeli**. Böylece jüri ve işverenler projeni senin bütçene bağlı kalmadan deneyebilir. Ücretsiz katmanlar ve yerel modeller için Ek A'ya bak.
- **Anahtarlar:** API anahtarları asla repoya girmez. `.env.example` dosyası koy, `.env` dosyası `.gitignore`'da olsun. Demo sitesinde anahtarları platformun "secrets" alanında sakla.

4. Takvim ve Kilometre Taşları

Proje dersler ilerledikçe büyür. Müfredat çalışma dosyasında 7. ve 9. ders için yeni başlık önerileri bulunur; proje belgelerindeki konu eşlemeleri bu önerilerin kesinleştiği anlamına gelmez. Her projenin gereksinimi kendi 7. bölümünde yazılıdır. Her kilometre taşı, o tarihe kadar işlenen derslerin konularını kullanır.

Kilometre taşı	Tarih	O tarihe kadar işlenen dersler	Bitmiş sayılır, eğer...
M0 · Başlangıç	2-5 Ekim	1-7 (LLM, tools, ReAct, smolagents, MCP, RAG)	Proje seçildi; GitHub deposu açıldı; veri indirildi; README'de problem tanımı ve 10 örnek kullanıcı sorusu var; LLM anahtarı alındı; deploy hesabı açıldı (Ek A).
M1 · Tek agent	5-11 Ekim	+ 8 Multi-agent; 9 Agentic RAG (Data Agent başlığı öneri)	Tek agent ≥ 4 tool ve bilgi katmanı ile CLI'da veya notebook'ta çalışıyor; baseline ölçüldü .
M2 · Sistem	12-18 Ekim	+ 10 Fine-tuning, 11 Evaluation & Observability	Multi-agent orkestrasyon, MCP server ve arayüz çalışıyor; trace'ler görünüyor.
M3 · Üretime hazır	19 Ekim → teslim	+ 12 Responsible AI & güvenlik	Golden set eval, guardrail ve injection testleri, deploy, dokümantasyon tamam; TVT/Medium yazısı yayında.
Son teslim	6 Kasım 2026, 23:59		Teslim formu MultiGroup duyurusuyla paylaşılır.

⚠ Bu akşam yap: HF Spaces'te ücretsiz demo açabilmek için hesabın **en az 30 günlük** olmalı. 2 Ekim'de açılan hesap ancak 1 Kasım'da uygun olur; o tarihte teslimde yalnızca beş gün kalır. HF hesabın yoksa hemen aç ya da varsayılan yol olan Streamlit Community Cloud'u kullan.

Kilometre taşları rehber niteliğindedir; yalnızca son teslim puanlanır. Ama M1'i 11 Ekim'e kadar bitirmeyenlerin işi son haftada katlanarak büyüyor. **Önce uçtan uca çalışan en basit sürümü yap, sonra büyüt.**

5. Repo Yapısı

Aşağıdaki yapı bir öneri. Klasörlerin hepsi zorunlu değil, ama aynı işlev görecek bir düzen bekliyoruz.

```

proje-adi/
├── README.md
├── .env.example
├── .gitignore
├── requirements.txt          # veya pyproject.toml
├── config.py                # model adları, yollar, iş kuralları, eşikler
├── data/                    # ham veri repoya girmez; indirme scripti girer
│   └── download.py
├── src/
│   ├── agents/              # her agent ayrı modül
│   ├── tools/               # tool tanımları + testleri
│   ├── mcp_server/          # kendi MCP server'ın
│   ├── knowledge/           # indeksleme, retrieval, semantic layer
│   ├── guardrails/          # PII, injection, kapsam kontrolleri
│   └── orchestrator.py
├── app.py                   # Gradio / Streamlit arayüzü
├── eval/
│   ├── golden_set.jsonl     # ≥25 örnek: girdi, beklenen çıktı/davranış, etiket
│   ├── injection_tests.jsonl
│   ├── run_eval.py
│   └── results/             # baseline ve final sonuç tabloları
├── notebooks/               # keşif ve deneyler
├── docs/                    # mimari diyagram, karar kayıtları, ekran görüntüleri
└── tests/                   # tool ve guardrail birim testleri

```

README'de bulunması gerekenler

- Proje başlığı ve tek cümlelik açıklama
- Çözülen problem, hedef kullanıcı ve neden agent gerektiği
- **Demo linki** ve ekran görüntüsü ya da GIF
- **TVT/Medium yazısının linki**
- Mimari diyagram (agent'lar, tool'lar, veri akışı)
- Kullanılan teknolojiler ve seçim gerekçeleri
- Veri kaynakları ve lisansları
- **Eval sonuç tablosu** (baseline vs final)
- Yerel kurulum adımları (temiz bir makinede 10 dakikada çalışmalı)
- Bilinen sınırlamalar ve yol haritası
- Repo yapısı ve iletişim bilgisi

6. Repoda Yazılı Olarak Cevaplanması Gereken Sorular

Bu soruların cevapları README'de veya docs/ altında yazılı olmalı. Mülakatta da sana tam olarak bunlar sorulacak.

1. **Problem:** Hangi problemi, kimin için çözüyorsun? Başarıyı neyle ölçüyorsun?

2. **Neden agent:** Sabit bir workflow veya tek LLM çağrısı neden yetmiyor? Agent'ın getirdiği ek maliyet ve karmaşıklığa değiyor mu?
3. **Mimari:** Agent'ları nasıl böldün, orkestrasyon nasıl çalışıyor? Hangi framework'ü neden seçtin?
4. **Tool tasarımı:** Tool'ların sınırlarını nasıl belirledin? Hangi tool okuma, hangisi yazma yapıyor? Yazma aksiyonlarını nasıl korudun?
5. **Bilgi katmanı:** Chunking, embedding, retrieval ve/veya semantic layer kararların neler? Retrieval kalitesini nasıl ölçtün?
6. **Baseline ve final:** Baseline'ın ne (ör. tool'suz tek LLM çağrısı ya da naif RAG) ve skoru kaç? Final sistemin skoru kaç? Fark hangi bileşenden geliyor (ablation)?
7. **Hata analizi:** En az 5 başarısız örnek; her biri neden başarısız oldu, ne değiştirdin, sonrasında ne oldu?
8. **Maliyet ve gecikme:** Bir görev ortalama kaç adım, kaç token, kaç saniye ve ne kadar maliyet tutuyor? Nasıl düşürdün?
9. **Güvenlik:** Hangi saldırıları denedin, kaçını engelledin? Hangi riskler hâlâ açık?
10. **Canlıya çıkış:** Bu sistem gerçek bir şirkette canlıya nasıl çıkar? Hangi metrikler hangi eşiklerle izlenmeli, ne zaman alarm çalmalı?

7. Değerlendirme (Eval) Nasıl Kurulur

Agent projelerinde "denedim, çalışıyor" bir ölçüm değildir. Asgari olarak şunları bekliyoruz:

- **Golden set:** En az 25 örnek. Kolay, çok adımlı, belirsiz, kapsam dışı ve saldırı örneklerinden oluşan karışık bir set kur. Her örnekte beklenen cevap ya da beklenen davranış yazılı olsun (ör. "şu tool'u çağırmalı", "reddetmeli", "şu kaynağı göstermeli").
- **Otomatik metrikler:** Proje dokümanındaki metrikler (sayısal doğruluk, tool çağrısı doğruluğu, SQL sonucu eşleşmesi, ranking metrikleri vb.).
- **LLM-as-judge:** Serbest metin cevaplarında sadakat (faithfulness) ve yardımcı olma puanı. Hakem prompt'unu repoda paylaş, en az 10 örnekte hakemin kararını elle kontrol et ve uyum oranını yaz.
- **Baseline vs final tablosu:** Aynı golden set üzerinde en az iki satır. İdeali her ana bileşeni (RAG, critic, semantic layer...) tek tek çıkarıp etkisini ölçmektir (ablation).
- **Tekrarlanabilirlik:** `python eval/run_eval.py` komutu sonuç tablosunu yeniden üretebilmeli. Model adı ve temperature değeri sonuç dosyasına yazılsın.

8. Güvenlik ve Etik Kurallar (Tüm Projeler)

- **Gerçek kişisel veri kullanma.** Yalnızca açık veri, anonim veri veya sentetik veri. Sağlık projesinde gerçek hasta bilgisi kesinlikle yasak (KVKK).
- **Dışa etki eden ya da kalıcı kaydı değiştiren aksiyonlar** (iade/talep oluşturma, kayıt güncelleme, mesaj gönderme, para hareketi) **insan onayı olmadan çalışmaz.** Onay arayüzde açıkça görünmeli. Yalnızca yerel bir dosya üreten ya da insana devreden tool'lar bu kapsamda değildir.

- **Yetkilendirme kodda yapılır, prompt'ta değil.** "Model başka müşterinin verisini göstermesin" kuralı tool katmanında uygulanmalı.
- **Dolaylı prompt injection'ı test et.** Dokümanın, yorumun ya da API cevabının içine gizlenmiş talimatlar modelin davranışını değiştirmemeli. En az 5 saldırı örneğini `eval/injection_tests.jsonl`'e koy.
- **Alan sınırlarını koru.** Finans projesi yatırım tavsiyesi, sağlık projesi tanı/tedavi önerisi vermez. Bu sınırlar test edilmiş olmalı.
- **Log'lar da veridir.** Trace'lere sır, anahtar veya hassas bilgi yazma.

9. Ortak Değerlendirme Rubriği

Üç projede kriterler ve ağırlıklar aynıdır. "Tam puan" tanımlarının projeye özel ayrıntıları her proje dokümanının 12. bölümünde yer alır.

Kriter	Ağırlık	Tam puan için özet
Problem tanımı & iş kurgusu	10	Gerçekçi persona, ölçülebilir hedef, "neden agent" ölçümle gerekçelendirilmiş
Agent & tool tasarımı	20	≥4 iyi tanımlanmış tool, sağlam hata yönetimi, döngü koruması, okuma/yazma ayrımı
Bilgi katmanı (RAG / veri / semantic layer)	15	Kaynak gösteren, ölçülmüş retrieval; kararlar gerekçeli
Multi-agent orkestrasyon & MCP	10	Net rol ayrımı, çalışan orkestrasyon, kendi MCP server'ı
Değerlendirme & observability	15	≥25 örnek golden set, baseline vs final, hata analizi, trace + maliyet/gecikme
Güvenlik & Responsible AI	10	Injection testleri, HITL, yetkilendirme, alan sınırları test edilmiş
Deploy & kullanıcı deneyimi	10	Çalışan public demo, anlaşılır arayüz, ara adımlar görünür
Dokümantasyon, repo & sunum	10	README eksiksiz, §6 soruları cevaplanmış, düzenli commit geçmişi, TVT ya da Medium'da yayınlanmış teknik yazı (§12)
Bonus	+10	Projenin İleri seviye maddeleri ya da §10'daki maddeler. Kaliteli ve ölçülmüş her madde için +2–4 puan, toplam en fazla +10

Her proje üç seviyede tanımlanmıştır:

- **Temel:** §3'teki asgari sürüm. Eksiksiz tamamlanan bir Temel proje yaklaşık **50–60 puan** hedefler. **50 puan resmî geçme eşiği değildir;** geçme eşiği ve sertifika koşulları MultiGroup duyurusuyla netleşir.
- **Hedef:** §3'ün tamamı. Portföyde güçlü duracak sürümdür ve 75–90 puan bandını hedefler.
- **İleri:** Seni öne çıkaran sürüm; bonus puanlar buradan gelir.

Önce Temel'i uçtan uca bitir, sonra Hedef'e büyüt. Yarım kalmış bir Hedef sürüm, eksiksiz bir Temel sürümden daha az puan alabilir.

10. Olsa Güzel Olur (Bonus)

- **Fine-tuning (Ders 10):** Router, intent sınıflandırıcı veya kriter sınıflandırıcı gibi dar bir görev için küçük bir modeli ince ayarla. Büyük modellerle maliyet/kalite karşılaştırmasını göster.
- **Knowledge graph (Ders 7):** Varlıklar ve ilişkiler üzerinden sorgu (şirket-rakip, ürün-kategori-satıcı, hastalık-biyobelirteç-ilaç).
- **Monitoring ekranı:** Eval ve trace sonuçlarını zaman içinde gösteren basit bir dashboard.
- **CI:** Her push'ta tool testleri ve küçük bir eval alt kümesinin otomatik çalışması.
- **Docker:** `docker compose up` ile tek komutla ayağa kalkan sistem.
- **Bulut deploy:** Google Cloud Run veya benzeri bir platformda çalışan sürüm (Ders 11). Faturalandırma hesabı ve kredi kartı ister; tamamen isteğe bağlıdır.
- **Üst yönetim sunumu:** Teknik olmayan bir yöneticiye 5 slaytta değer önerisi, maliyet ve risk.
- **Demo videosu:** Projeni 2–3 dakikada gösteren bir YouTube ya da Loom videosu.

11. Yapay Zekâ Destekli Kodlama Hakkında

Kod yazarken AI asistanları kullanabilirsin. Bunu saklama, README'de belirt. Ancak:

- Repodaki her satırı açıklayabilmelisin. Mülakatta "bu fonksiyon ne yapıyor, neden böyle?" sorusunun cevabı "asistan yazdı" olamaz.
- Mimari kararlar, eval tasarımı ve hata analizi **senin** düşünce sürecini göstermeli.
- Başkasının projesini kopyalamak veya bir şablon repoyu olduğu gibi teslim etmek değerlendirme dışı kalmaya yol açar.

12. Teslim

- **Ne teslim edilir:** Public GitHub repo linki, teslim anındaki commit (ya da `v1.0` tag'i), TVT ya da Medium yazısının linki, demo linki (Hedef seviye). Demo videosu isteğe bağlıdır ve bonus puan alır (§10).
- **Nasıl:** MultiGroup'un duyuracağı teslim formu üzerinden. **Form bağlantısı bu pakette henüz yoktur; TVT ana sayfasını form olarak kullanma.**
- **Son tarih:** 6 Kasım 2026 Cuma, 23:59.
- **Sonuçlar:** Kazanan projeler 1 Aralık 2026'ya kadar açıklanır.
- **Ekip:** Bireysel çalışma mümkün. Üç kişilik üst sınır önerisi MultiGroup teyidi bekliyor; ekiplerde her üyenin katkısı commit geçmişinde ve README'de görünmeli.
- **Teslimden sonra:** Demo linki değerlendirme bitene kadar açık kalmalı. Uyuyan bir demoyu uyandırmak jüriye bir dakikadan uzun sürüyorsa README'deki ekran görüntüleri ve yazın yedek olur.

Teknik yazı (zorunlu)

Projeni Türkiye Veri Topluluğu (TVT) yayınında ya da kendi Medium hesabında bir teknik yazıyla anlat. Yazı her seviyede zorunludur ve teslim formunda linkiyle birlikte istenir. Yazısı olmayan teslim eksik teslim sayılır.

- **Ne anlatmalı:** Çözdüğün problem ve kimin için çözdüğün; mimari diyagram; tool ve bilgi katmanı kararların; baseline ile final sistemin karşılaştırma tablosu; bir trade-off ("X'i denedim, Y yüzünden Z'ye geçtim"); sistemin hâlâ yapamadıkları.
- **Uzunluk ve dil:** 1.000–2.000 kelime, Türkçe ya da İngilizce. Kod parçaları kısa olsun; tam kod repoda dursun.
- **Yayın:** Yazı en geç 6 Kasım 2026 23:59'da yayında ve **herkese açık** olmalı (Medium'da üyelere özel yazı jüri tarafından okunamayabilir). TVT yayınına göndermek istiyorsan editör kanalının organizatör tarafından duyurulmasını bekle; kanal açıklanırsa yazını en geç **30 Ekim**'de gönder. Editör kanalı açıklanmaz veya yazı 6 Kasım teslim tarihine kadar yayınlanmazsa aynı yazıyı Medium'da herkese açık yayımla ve o linki teslim et. TVT ana sayfası bir yazı gönderim sayfası değildir. Teslimden sonra yazıda yalnızca yazım düzeltmesi yapılabilir.
- **Ekipler:** Ekip başına tek ortak yazı yeterli; bütün ekip üyeleri yazar olarak yazıda listelenir.
- **Bağlantılar:** Yazıda repo ve (varsa) demo linki; README'de yazının linki.
- **Kurallar:** API anahtarı, gerçek kişisel veri ya da ham müşteri kaydı paylaşma. Ekran görüntülerinde de .env içeriği, ham müşteri/sipariş kimliği ya da e-posta görünmesin. Yazının sahibi sensin; yapay zekâ desteği kullandıysan bunu belirt (§11).

Bu yazı mülakatta da en güçlü kanıtın olur: görüşmeden önce linkini paylaşabilirsin.

13. Mülakatta Projeni Anlatmak: Ortak Çerçeve

Her proje dokümanının 14. bölümünde projeye özel bir pitch ve olası sorular var. Ortak çerçeve şu:

1. **60 saniyelik pitch:** Problem → kimin için → ne yaptın → bir sayı → bir zorluk ve nasıl çözdüğün.
2. **Üç sayı:** Baseline'a göre iyileşme, güvenlik testi başarı oranı, görev başı maliyet/gecikme. Sayısı olmayan proje "demo"dur, sayısı olan proje "mühendisliktir".
3. **Bir trade-off hikâyesi:** "X'i denedim, Y yüzünden Z'ye geçtim." Mülakatçılar en çok bunu dinler.
4. **Bir sınırlama:** Sistemin hâlâ neyi yapamadığını ve sırada ne olduğunu dürüstçe söyle.
5. **Canlı demo planı:** İnternet ya da API yavaşlarsa diye 2 dakikalık kayıtlı video yedeğin olsun.

Ek A. Ücretsiz / Düşük Maliyetli Teknoloji Seçenekleri

Aşağıdaki bilgiler 1 Ekim 2026'da resmî kaynaklardan doğrulandı. Ücretsiz katman koşulları sık değişir; projeye başlamadan önce kendi hesabında kontrol et.

Katman	Önerilen ücretsiz seçenek	Notlar
LLM (varsayılan)	Google Gemini API ücretsiz katmanı (AI Studio anahtarı)	Function calling destekli; OpenAI uyumlu endpoint'i de var. Güncel istek limitlerini AI Studio → <i>Rate limits</i> ekranında gör. Ücretsiz katmanda gönderilen içerik ürün iyileştirmede kullanılabilir, bu yüzden gizli veya kişisel veri gönderme .
LLM (yedek)	Groq ücretsiz katmanı	Modellerin tool calling desteği var. Dakika başına token limiti düşük, bu yüzden RAG bağlamını kısa tut.
LLM (tamamen yerel, anahtarsız)	Ollama + qwen3 (4B/8B) veya llama3.2:3b	Tool calling destekli küçük modeller. Kalite düşer ama internet ve kota sorunu yaşamazsın; eval'de karşılaştırma satırı olarak da değerli.
Deneme amaçlı	Hugging Face Inference Providers, OpenRouter :free modeller	Aylık kredi ve günlük istek sayısı çok düşük. Varsayılan yapma.
Embedding	Demo/deploy için hafif <code>intfloat/multilingual-e5-small</code> ; yerel geliştirmede daha güçlü BAAI/bge-m3	İkisi de çok dilli ve yerelde çalışır. <code>bge-m3</code> büyük bir model; Streamlit Community Cloud gibi düşük RAM'li ücretsiz ortamlarda demoyu çökterebilir. İndeks ve sorgu aynı embedding modeliyle üretilmeli. Deploy edeceğin modeli baştan seç; indeksi yerelde üretip deploy'a hazır dosya olarak koy. Gemini embedding ücretsiz katmanda yok. Reranker için BAAI/bge-reranker-v2-m3 (deploy'da isteğe bağlı).
Vektör DB	Chroma, Qdrant (yerel mod) veya LanceDB	Üçü de sunucu kurmadan dosya üzerinde çalışır.
Observability	Arize Phoenix (yerel, ücretsiz; Elastic License 2.0) veya Langfuse Cloud Hobby	smolagents için <code>openinference-instrumentation-smolagents</code> ; Google ADK yerleşik OpenTelemetry desteğiyle gelir.
Eval	DeepEval	RAGAS kullanılabilir ama bakımı yavaşladı. Google ADK kullananlar <code>adk eval</code> komutunu da kullanabilir.
Deploy (varsayılan)	Streamlit Community Cloud	Ücretsiz, public GitHub reposundan deploy edilir. Kullanılmayınca uykuya geçer; demo öncesi uyandır.
Deploy (alternatif)	Hugging Face Spaces, Gradio + ZeroGPU	Dikkat: HF'de Gradio/Docker Space açmak artık ücretli plan istiyor. Ücretsiz hesaplar en fazla 2 Gradio ZeroGPU Space açabilir. Hesabın en az 30 günlük ve e-postası doğrulanmış olmalı. HF hesabın yoksa bugün aç , yoksa teslimden önce uygun hâle gelmeyebilir. Streamlit'i HF'de çalıştırmak Docker ister, yani ücretli plan gerekir.
Deploy (üçüncü seçenek)	Render ücretsiz katmanı	15 dakika kullanılmayınca uyur, ilk istekte yaklaşık 1 dakikada uyanır.

MCP Python SDK sürüm uyarısı

mcp paketinin 2.x sürümünde API değişti ve framework'ler henüz aynı sürümde buluşmuyor. Kurulumda saatler kaybetmemek için:

- **smolagents veya LangGraph** (`langchain-mcp-adapters`) kullanıyorsan: `requirements.txt`'ye `mcp[ws]<2` yaz. Server'ı `from mcp.server.fastmcp import FastMCP` ile yaz.
- **LlamaIndex** (`llama-index-tools-mcp`) veya **fastmcp 4.x** kullanıyorsan: `mcp>=2`. 2.x'te `FastMCP` sınıfının adı `MCPServer` oldu (`from mcp.server.mcpserver import MCPServer`). İnternetteki eski örnekler bu yüzden hata verebilir.
- **Google ADK** iki sürümle de çalışır.

- En sorunsuz yol: MCP server'ını ayrı bir süreçte (stdio transport) çalıştır, gerekirse ayrı sanal ortamda.
- Bir kütüphanenin **hazır** MCP server'ını olduğu gibi kullanmak §3 #4'ü karşılamaz. Server'ı sen yazmalısın.

Ek B. Yardımcı Kaynaklar

- Bootcamp reposu: <https://github.com/multigroupco/AI-Agents-Bootcamp> (ders slaytları, mimari diyagramlar, Shared/docs/curriculum_deep_dive.md)
- Hugging Face Agents Course: <https://huggingface.co/learn/agents-course>
- Model Context Protocol dokümantasyonu: <https://modelcontextprotocol.io>
- Anthropic, "Building effective agents": <https://www.anthropic.com/engineering/building-effective-agents>
- DMG Data Science Awesome: <https://github.com/Developer-MultiGroup/DMG-Data-Science-Awesome>

Başarılar! Takıldığında önce trace'e bak. Agent'ın hangi adımda neyi gördüğünü ve neye karar verdiğini görmek, hatanın yarısını çözer.